Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0141 | G01

BASES DE DATOS AVANZADAS

Laboratorio 5

Bases de Datos Distribuidas

Profesor:

Mag. Luis Carlos Salas

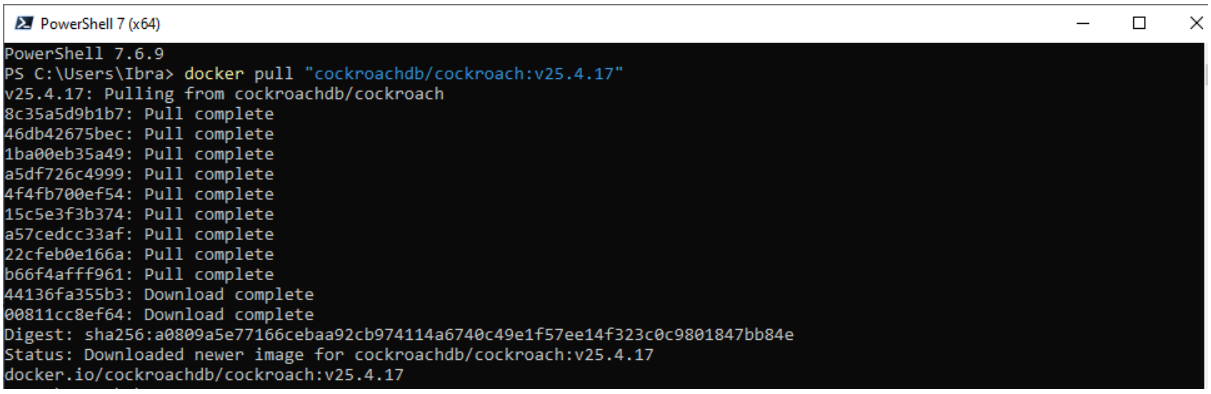
Javier Cruz Ruiz C02517

2 de Noviembre, 2026

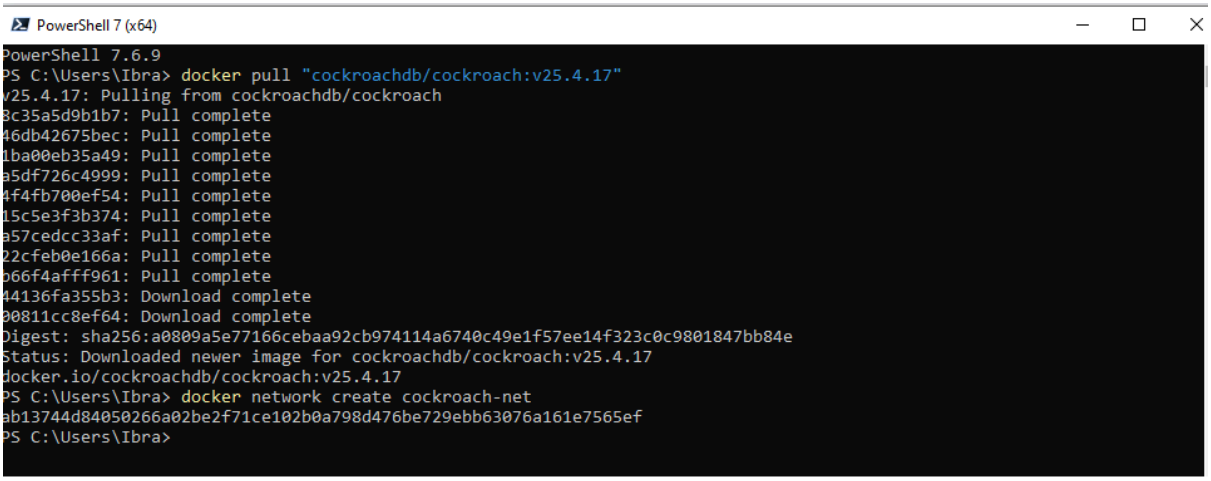
Creación del Cluster

1. Realice los siguientes pasos para realizar la habilitación de un cluster en CockroachDB. Hay que tener en cuenta que ejecutar varios nodos de un cluster en un solo host es útil sólo para probar CockroachDB, pero no es adecuado para producción.

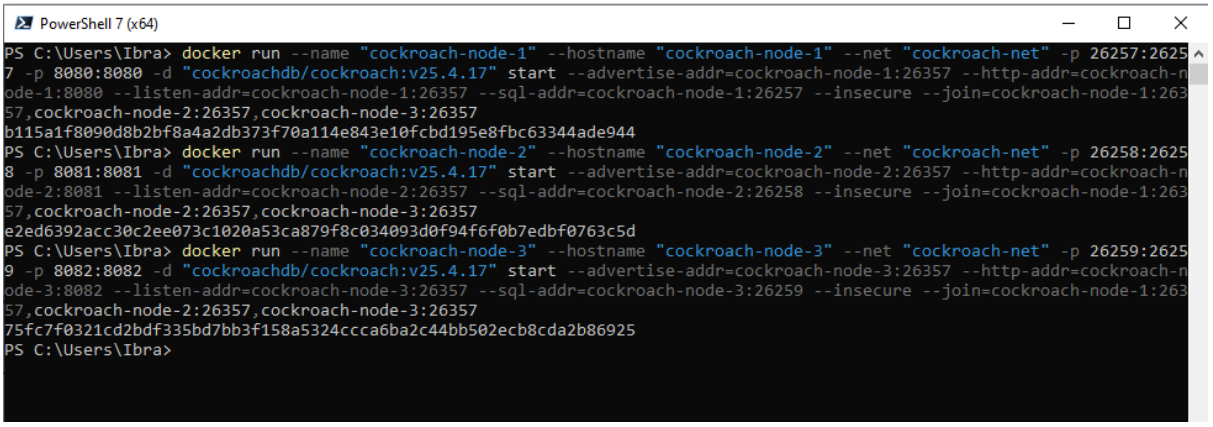
a. Ejecute el siguiente comando para descargar la imagen de CockroachDB 25.4.17.



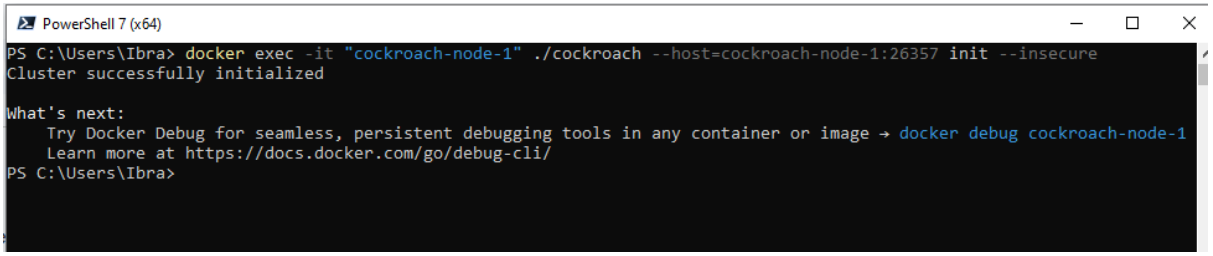
b. Ejecute el siguiente comando para crear una red interna, que se utilizará para conectar y comunicar los contenedores entre sí



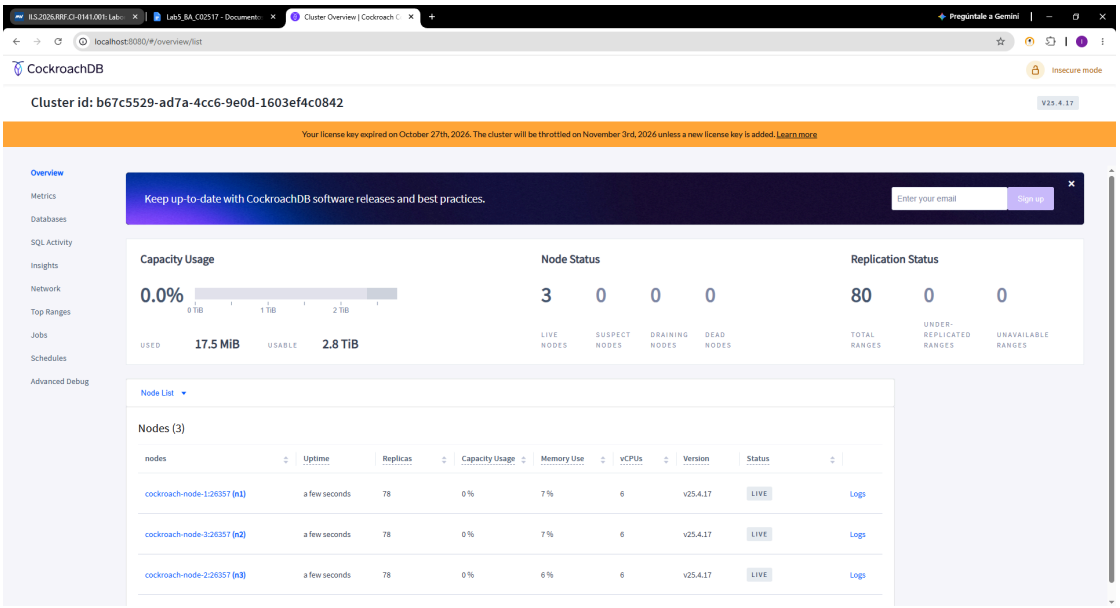
c. Ejecute los siguientes comandos para iniciar un clúster de tres nodos de CockroachDB, donde cada nodo se desplegará en un contenedor independiente, almacenará sus datos localmente y atenderá conexiones SQL y HTTP en un puerto único dentro de la red interna.



d. Ejecute el siguiente comando para realizar la inicialización del clúster. Este comando hace la inicialización desde el nodo “cockroach-node-1”, pero podría ejecutarse desde cualquier no.



e. Abra en el navegador la consola de administración del nodo 1 en el puerto 8080 utilizando la URL http://localhost:8080 y verifique en la pestaña de “Overview” que los tres nodos están en línea. Incluya una captura de pantalla.

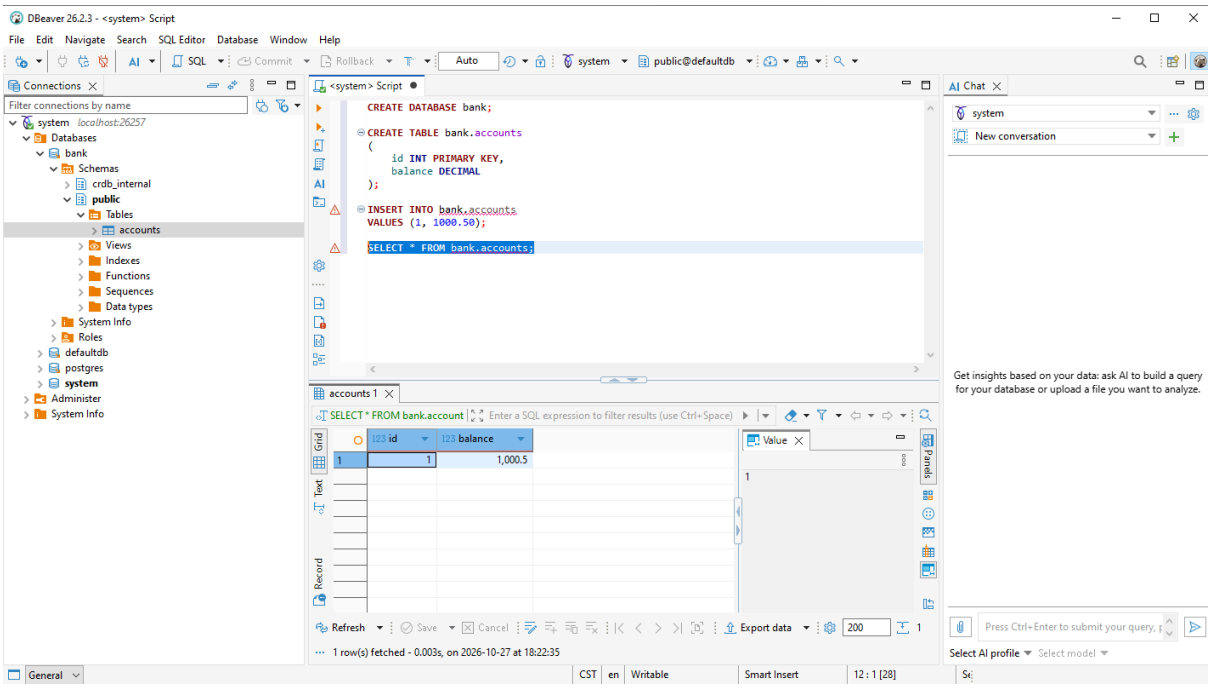


f. Conéctese al nodo 1 del cluster utilizando el puerto 26257, con el cliente "DBeaver Community", utilizando los siguientes pasos:

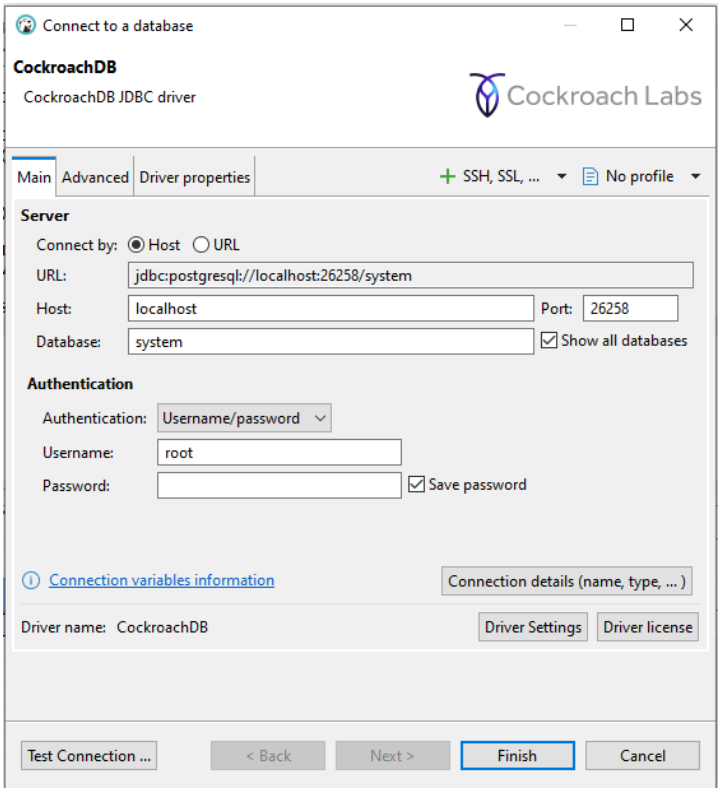


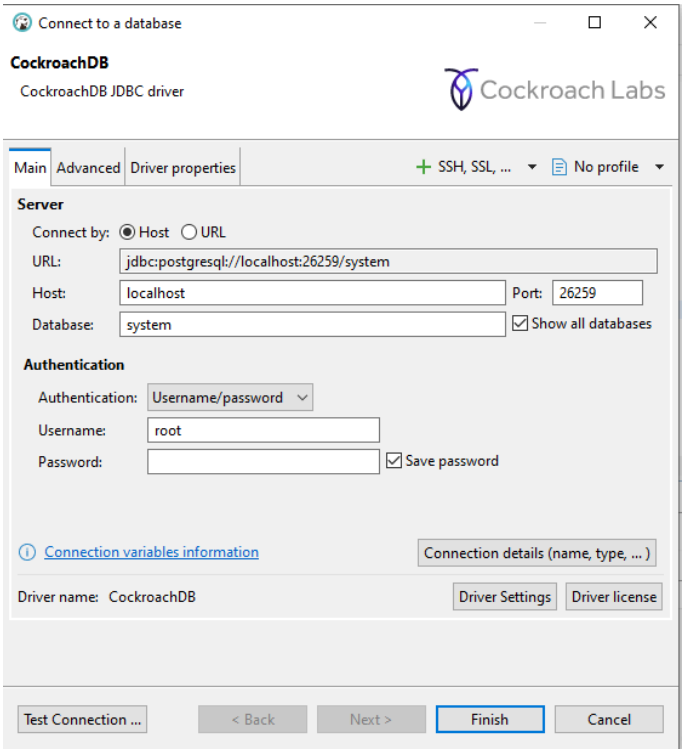
Funcionamiento del Cluster

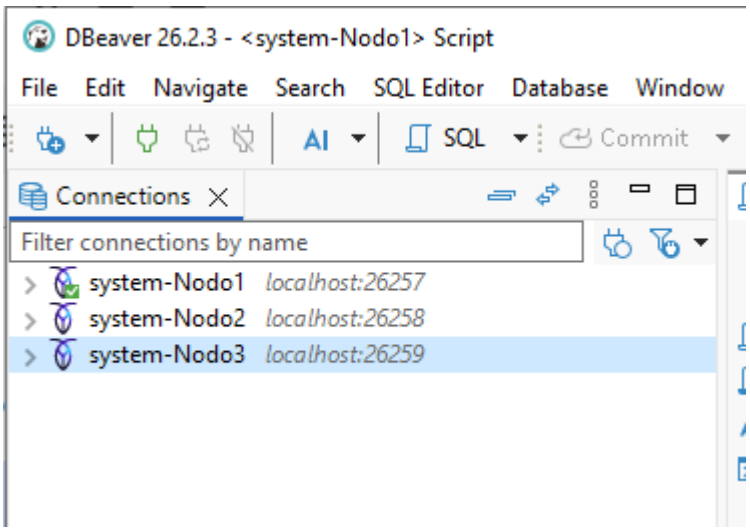
1. Cree la base de datos "bank" ejecutando el siguiente script en el nodo 1.



2. Conéctese al nodo 2 y al nodo 3 del cluster con el cliente "DBeaver Community", de la misma forma en que se conectó al nodo 1, sólo que utilizando el puerto 26258 para el nodo 2 y el puerto 26259 para el nodo 3, incluya una captura de pantalla para ambas conexiones.

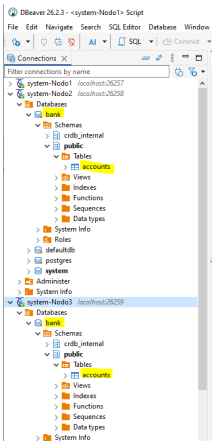

Connect to a database
CockroachDB
CockroachDB JDBC driver
Cockroach Labs
Main
Advanced
Driver properties
SSH, SSL, ...
No profile
Server
Connect by: Host URL
URL: jdbc:postgresql://localhost:26258/system
Host: localhost
Port: 26258
Database: system
Show all databases
Authentication
Authentication: Username/password
Username: root
Password:
Save password
Connection variables information
Connection details (name, type, ...)
Driver name: CockroachDB
Driver Settings
Driver license
Test Connection ...
< Back
Next >
Finish
Cancel

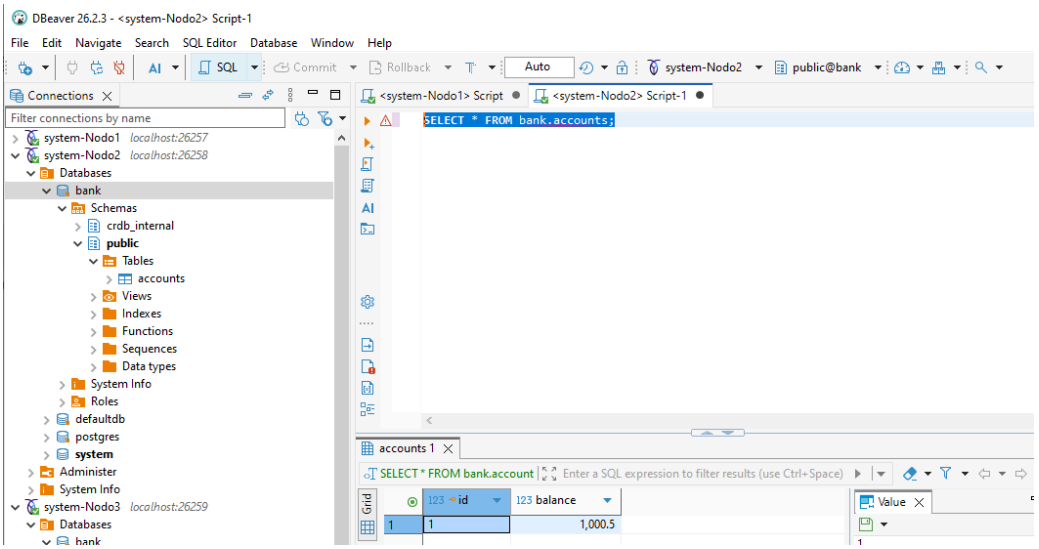

Connect to a database
CockroachDB
CockroachDB JDBC driver
Cockroach Labs
Main
Advanced
Driver properties
SSH, SSL, ...
No profile
Server
Connect by: Host URL
URL: jdbc:postgresql://localhost:26259/system
Host: localhost
Port: 26259
Database: system
Show all databases
Authentication
Authentication: Username/password
Username: root
Password:
Save password
Connection variables information
Connection details (name, type, ...)
Driver name: CockroachDB
Driver Settings
Driver license
Test Connection ...
< Back
Next >
Finish
Cancel

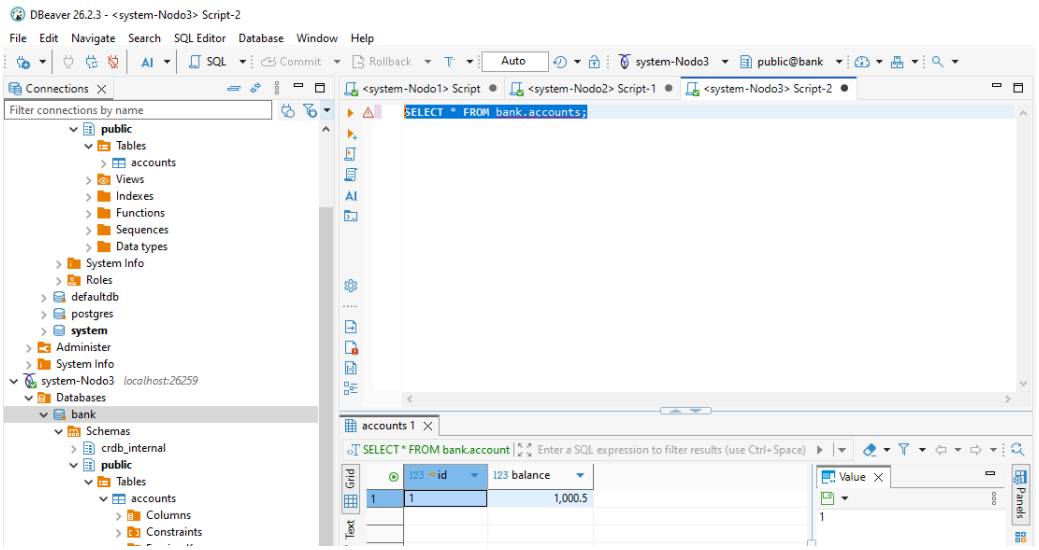


3. ¿Existe en el nodo 2 y en el nodo 3 la base de datos “bank”, la tabla “accounts” y los datos que se crearon en el paso anterior en el nodo 1? Explique lo que pudo haber sucedido utilizando el conocimiento del curso e incluya una captura de pantalla. Comente a qué principios de los DDMBS corresponde el comportamiento observado, considerando las diferentes promesas de transparencia vistas en el curso.

En el nodo 2 y 3 sí existe las base de datos “bank” con su respectiva tabla “accounts” y los datos insertados en el paso anterior. CockroachDB es una base de datos distribuida, esto significa que los datos se replican automáticamente, por esta razón ocurrió que lo hecho en el nodo 1 luego apareció en el nodo 2 y 3. El comportamiento corresponde a la transparencia de distribución, aunque los datos están en tres nodos, el usuario ve una sola base de datos lógica y el sistema se comporta como si fuera centralizado.



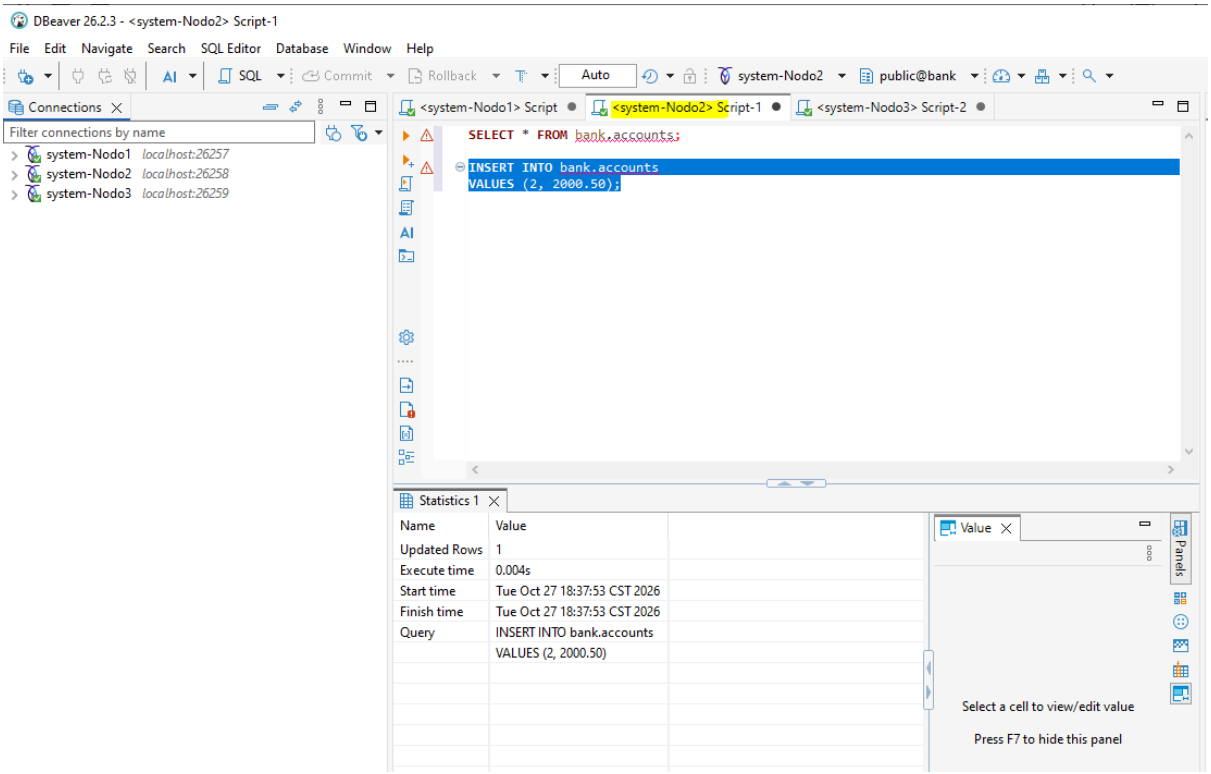




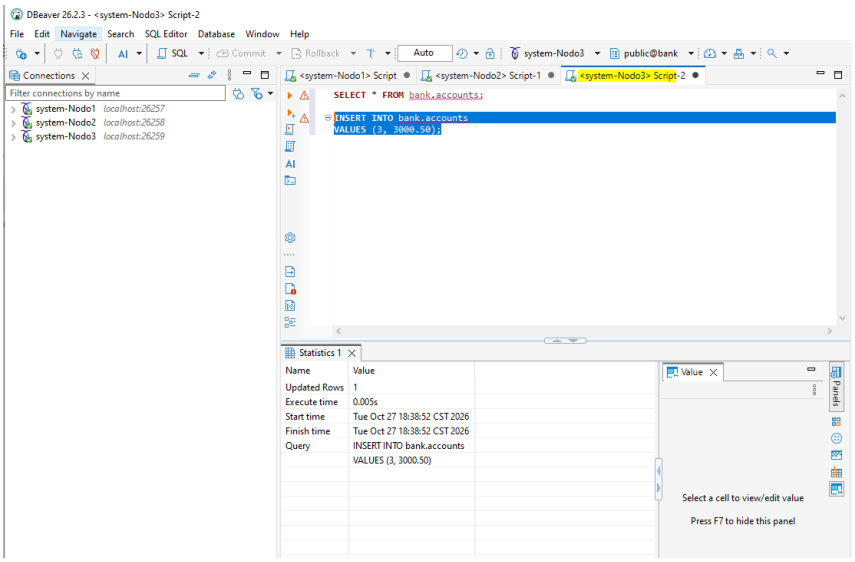
4. Habiendo observado el funcionamiento del DBMS CockroachDB, compárelo con el funcionamiento del DBMS SQL Server. Comente brevemente lo que hubiera tenido que hacer en SQL Server para tener un funcionamiento similar al de CockroachDB.

CockroachDB es distribuido de forma nativa, los nodos forman un clúster que replica los datos automáticamente, por lo que lo creado en el nodo 1 aparece en los demás. En SQL Server, cada instancia es independiente, para hacer algo similar habría que configurar manualmente la replicación, por ejemplo con publicador y suscriptores.

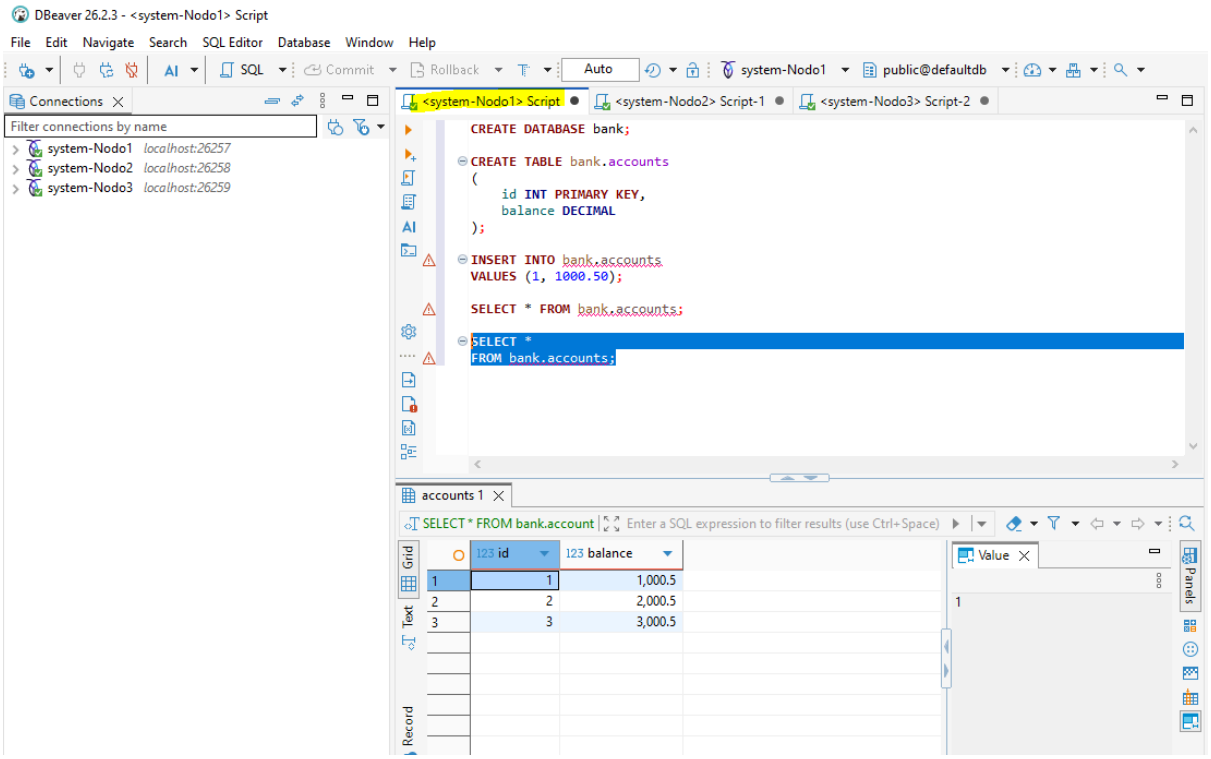
5. Estando conectado al nodo 2, inserte un registro en la tabla “accounts” en la base de datos “bank” utilizando el siguiente script. Incluya una captura de pantalla que muestre el resultado de la inserción y que la realizó conectado en el nodo 2.



6. Estando conectado al nodo 3, inserte un registro en la tabla “accounts” en la base de datos “bank” utilizando el siguiente script. Incluya una captura de pantalla que muestre el resultado de la inserción y que la realizó conectado en el nodo 3.

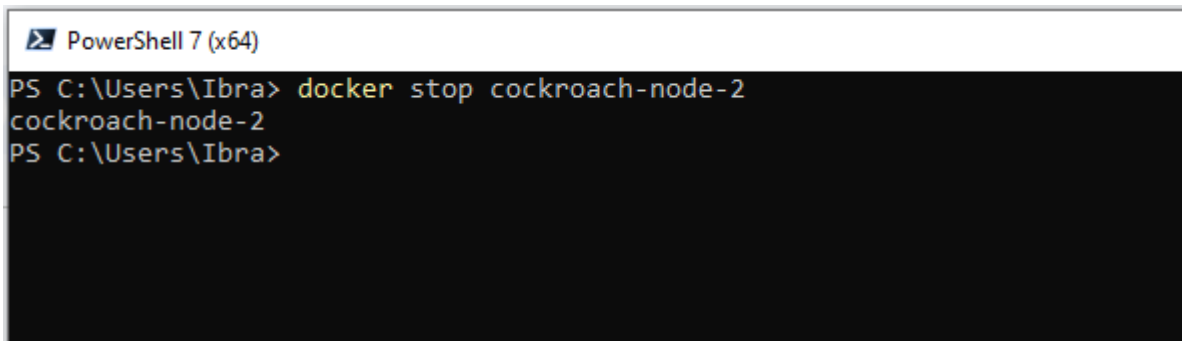


7. Estando conectado al nodo 1, consulte los datos en la tabla “accounts” en la base de datos “bank” utilizando el siguiente script. Incluya una captura de pantalla que muestre el resultado de la consulta y que la realizó conectado en el nodo 1.

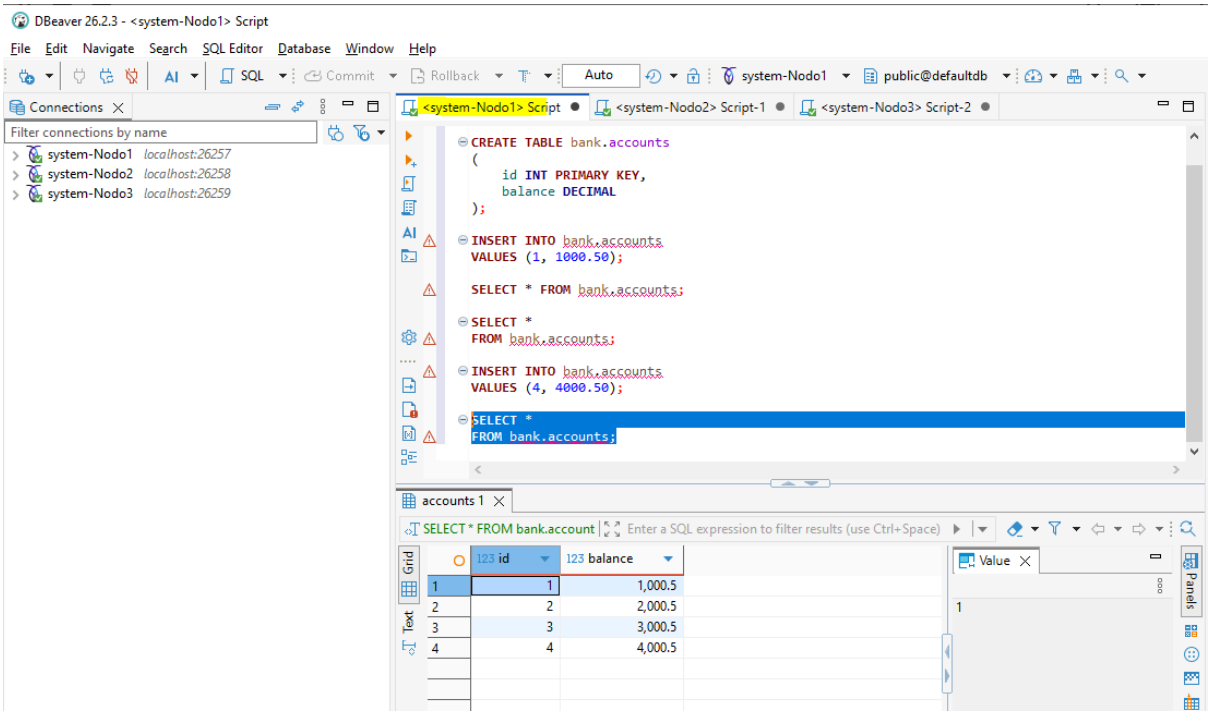


Funcionamiento de recuperación

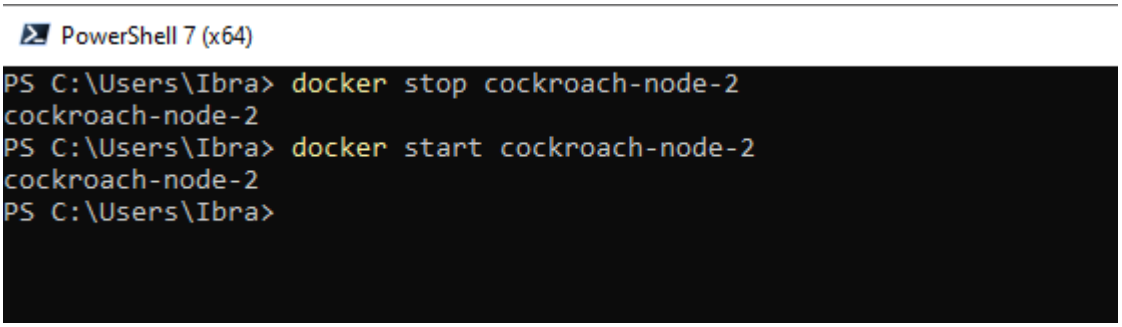
1. Ejecute el siguiente comando para detener el contenedor del nodo 2 del cluster.



2. Estando conectado al nodo 1, inserte un nuevo registro en la tabla “accounts” en la base de datos “bank” utilizando el siguiente script. Verifique que el registro con id = 4 esté como resultado.

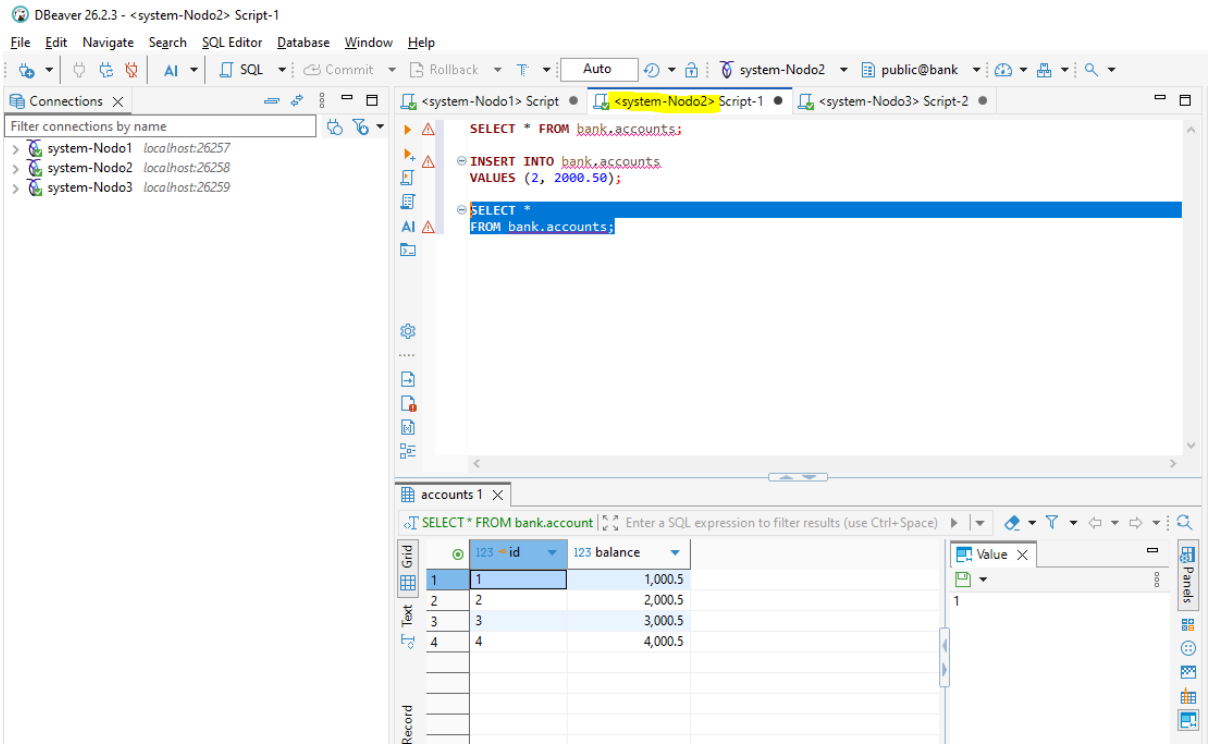


3. Ejecute el siguiente comando para iniciar nuevamente el contenedor del nodo 2 del cluster.



4. Espere mínimo 1 minuto, para permitir que el nodo 2 se reincorpore en el clúster.

5. Estando conectado al nodo 2, consulte si el registro con id = 4 está en la tabla “accounts” en la base de datos “bank” utilizando el siguiente script.

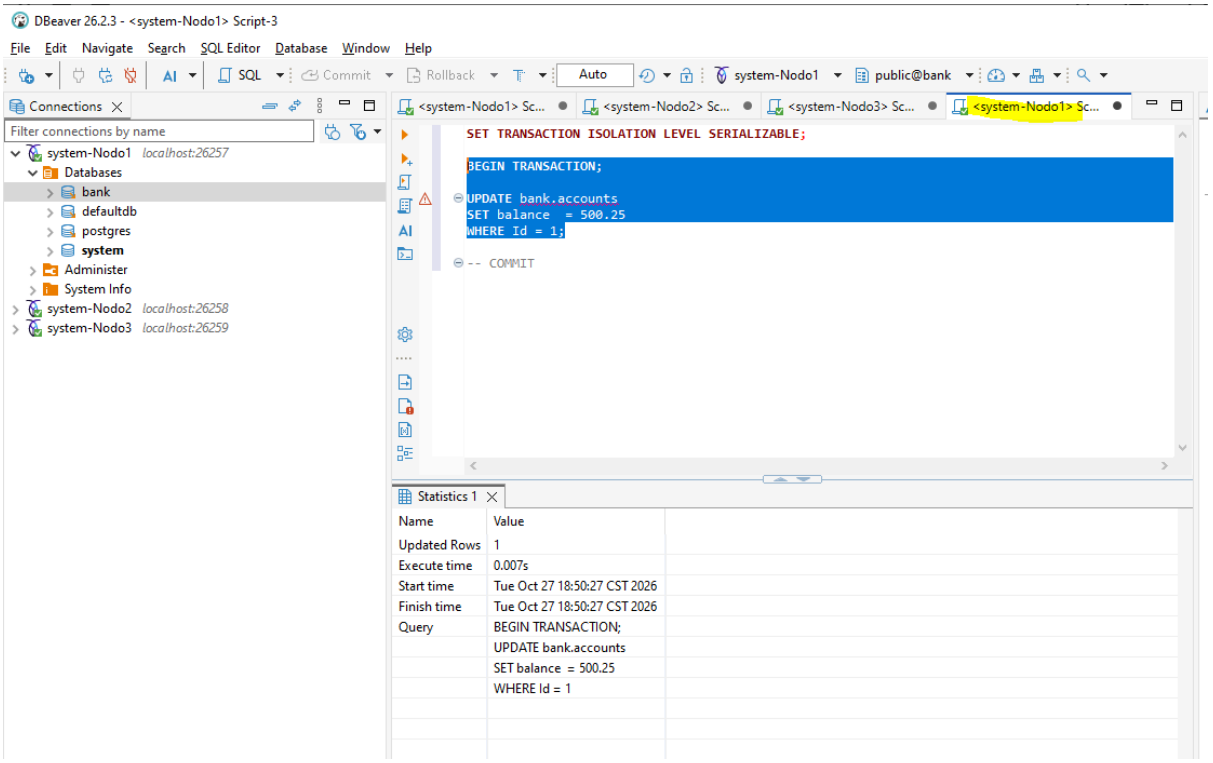


6. ¿Está el registro con id = 4 en la base de datos del nodo 2? Utilizando el conocimiento de recuperación de base de datos distribuidas visto en el curso, ¿Considera que el comportamiento observado es el esperado en un sistema gestor de bases de datos distribuidas? Justifique su respuesta.

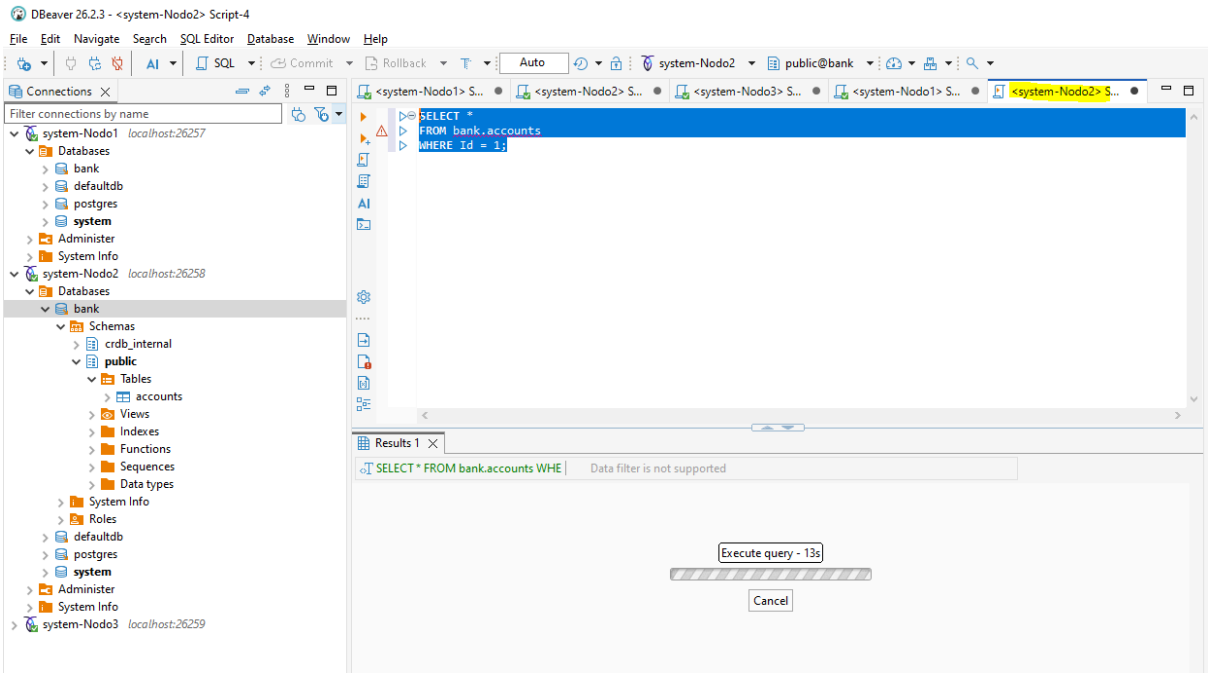
El registro con Id = 4 sí se encuentra en la base de datos del nodo 2. Este comportamiento es el esperado del DBMS, ya que mientras que el nodo 2 estaba detenido, los nodos 1 y 3 siguieron funcionando, así que el insert se ejecutó al recuperarse el nodo. El nodo 2 se "puso al día" con lo que se había perdido.

Funcionamiento de transacciones

1. Abra un nuevo query, establezca una conexión con el nodo 1 y ejecute el siguiente script (no debe ejecutar el comando COMMIT que está comentado, eso se le solicitará más adelante, la intensión en este punto es que la transacción este abierta):



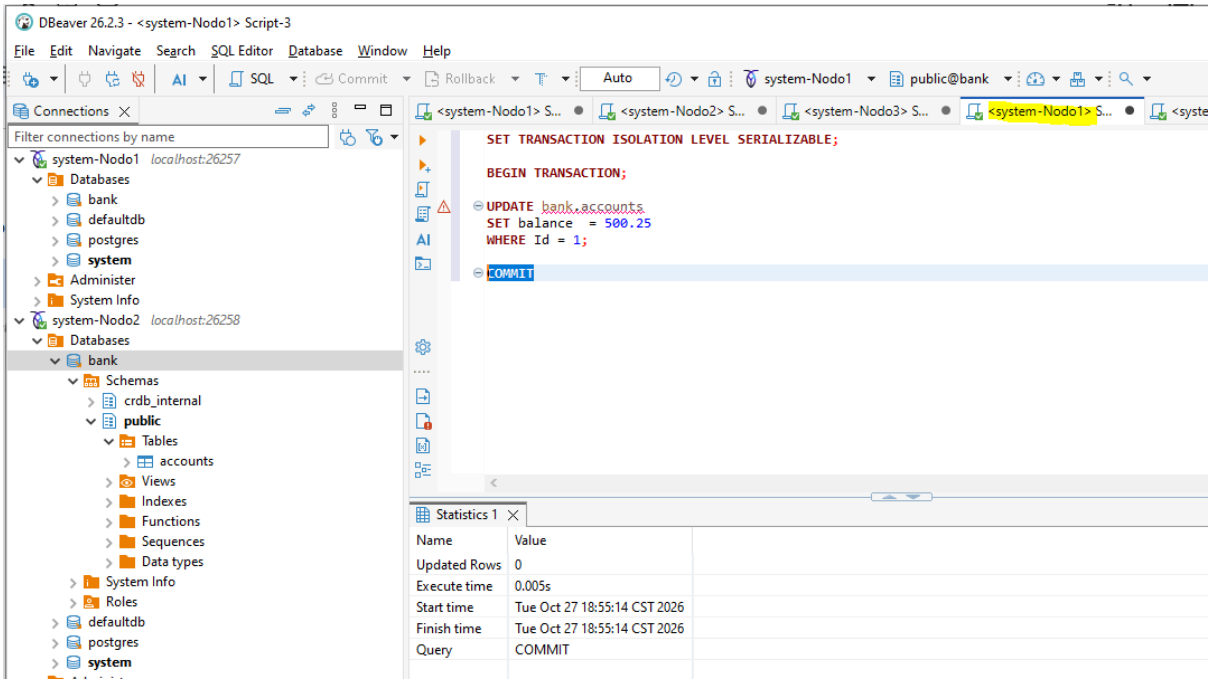
2. Abra un nuevo query, establezca una conexión con el nodo 2 y ejecute el siguiente script:

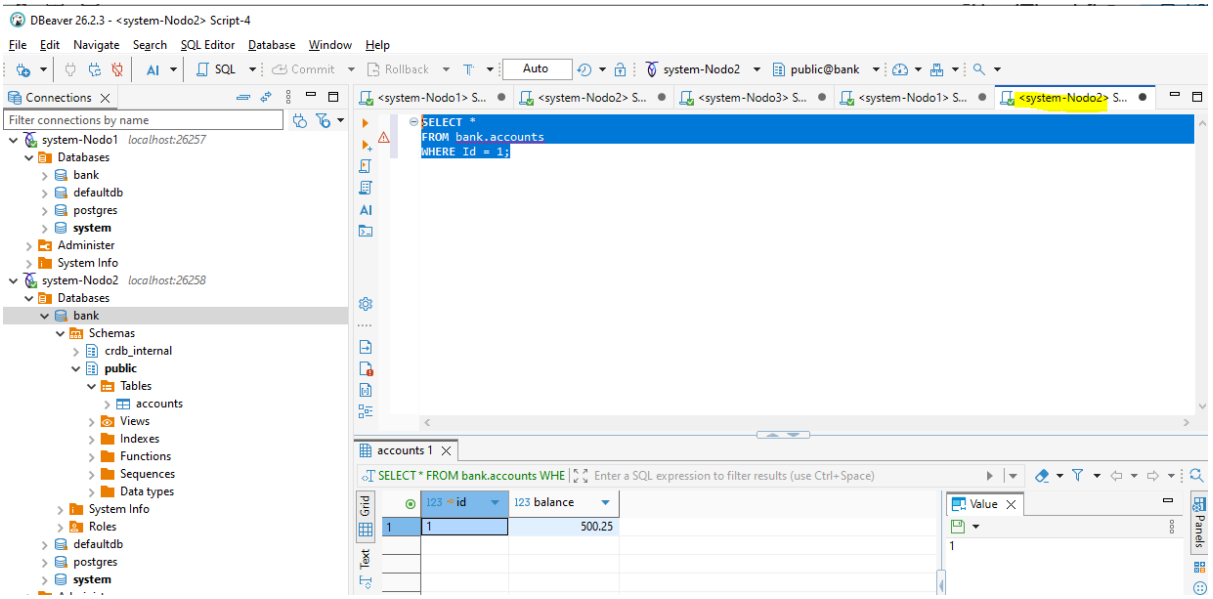


3. ¿Qué efecto observa sobre la ejecución del script anterior, sí se pudo o no se pudo obtener los datos del registro con Id=1 en el nodo 2? Explique las razones para el comportamiento observado.

Tras la ejecución del query anterior, no se logró obtener los datos del registro con Id = 1 en el nodo 2, de momento se queda cargando sin avanzar. Este es el comportamiento esperado, con el aislamiento SERIALIZABLE, una transacción no puede leer cambios que aún no están committeados, así que el nodo 2 espera a que la transacción termine con COMMIT o ROLLBACK.

4. Ejecute el COMMIT en el query que actualmente tiene la transacción abierta en el nodo 1 y conteste las siguientes consultas:





a. ¿Qué efecto tuvo el COMMIT sobre el query que estaba consultando los datos del registro con Id=1 en el nodo 2?

El COMMIT desbloqueó la fila con id = 1, así que el SELECT que estaba esperando en el nodo 2 se completó y devolvió el resultado.

b. ¿Qué valor de la columna "balance" que se obtuvo en el nodo 2, fue el mismo que se actualizó en el nodo 1?

El valor obtenido es el mismo que se actualizó en el nodo 1, no se mostró el valor anterior.

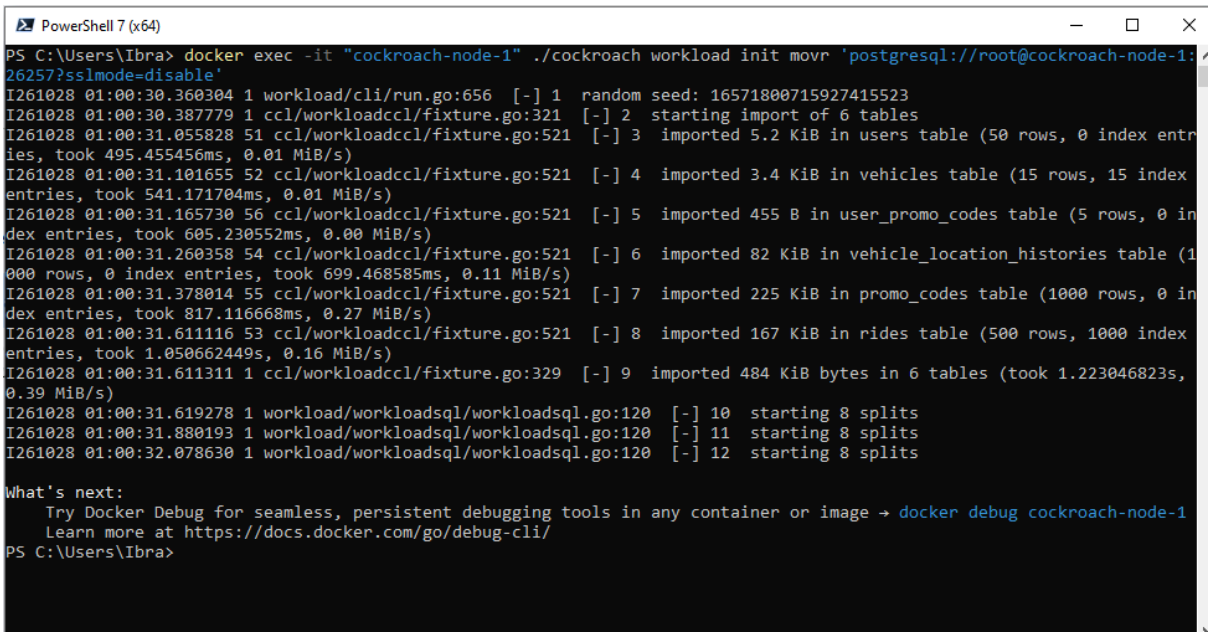
c. ¿Considera que el comportamiento observado es el esperado en un sistema gestor de bases de datos distribuidas? Justifique su respuesta.

Sí, es el comportamiento esperado en un DDBMS. El sistema se aseguró de que las transacciones se comportaran como si se ejecutaran una por una, de esta forma el nodo 2 no leyó datos sin confirmar y esperó hasta que se hizo el COMMIT. Luego, una vez committeado, el cambio se vio de en el otro nodo, lo que muestra consistencia entre réplicas y que el control de concurrencia funciona igual sin importar a qué nodo se conecte cada usuario.

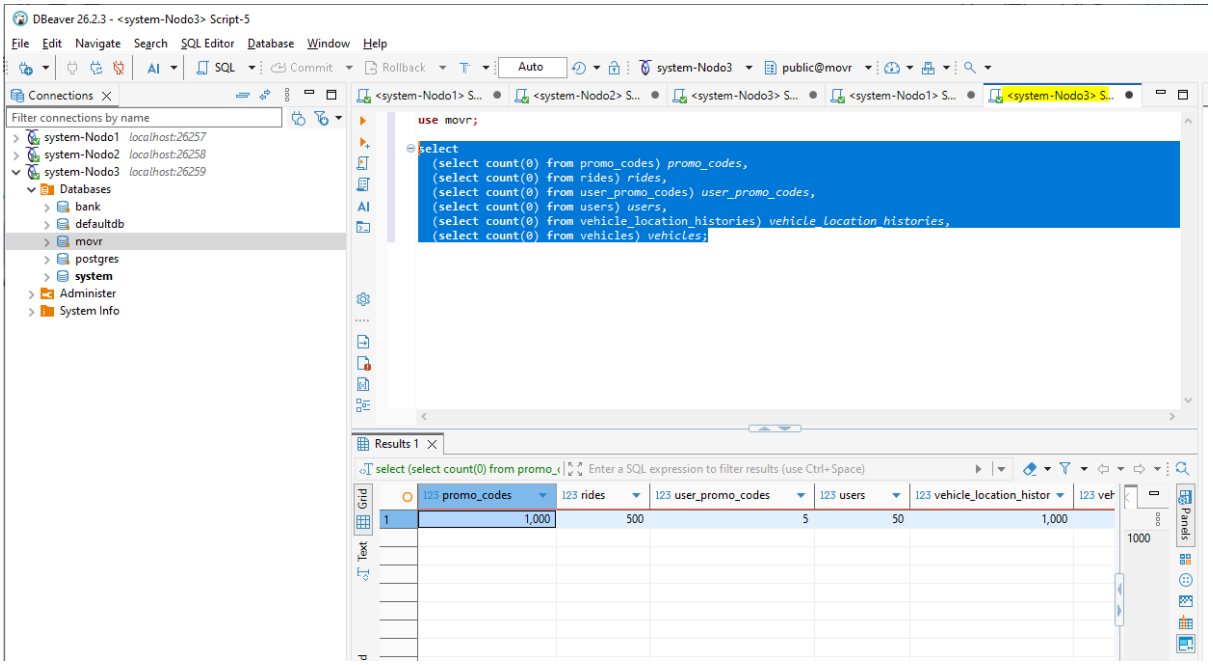
Simulación

1. Realice los siguientes pasos para realizar la simulación de ejecución de una aplicación contra el cluster. CockroachDB incluye un simulador de cargas de trabajo integradas para simular el tráfico de clientes. Se va a realizar la simulación de ejecución de una aplicación ficticia de uso compartido de vehículos. Puede ver los detalles de esta simulación en este enlace.

a. Ejecute el siguiente comando para iniciar la base de datos necesaria para la simulación.



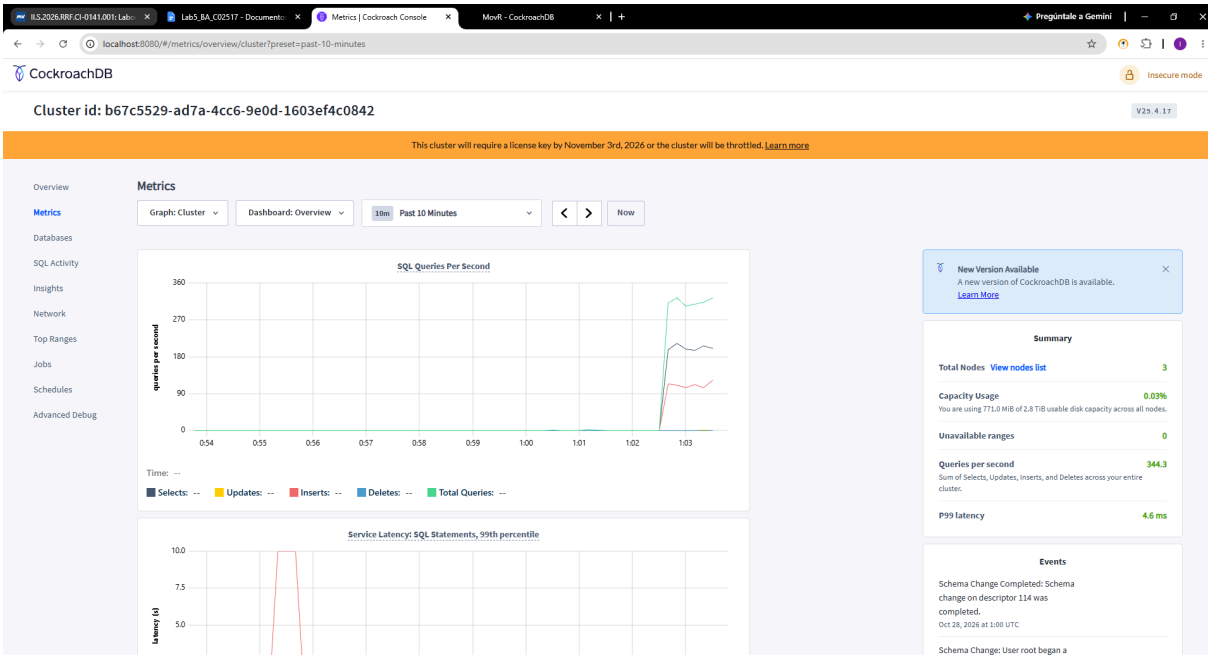
b. Estando conectado a cualquier nodo, ejecute el siguiente script para consultar los datos de la base de datos “movr” que se acaba de insertar. Incluya una captura de pantalla que muestre el resultado de la consulta.



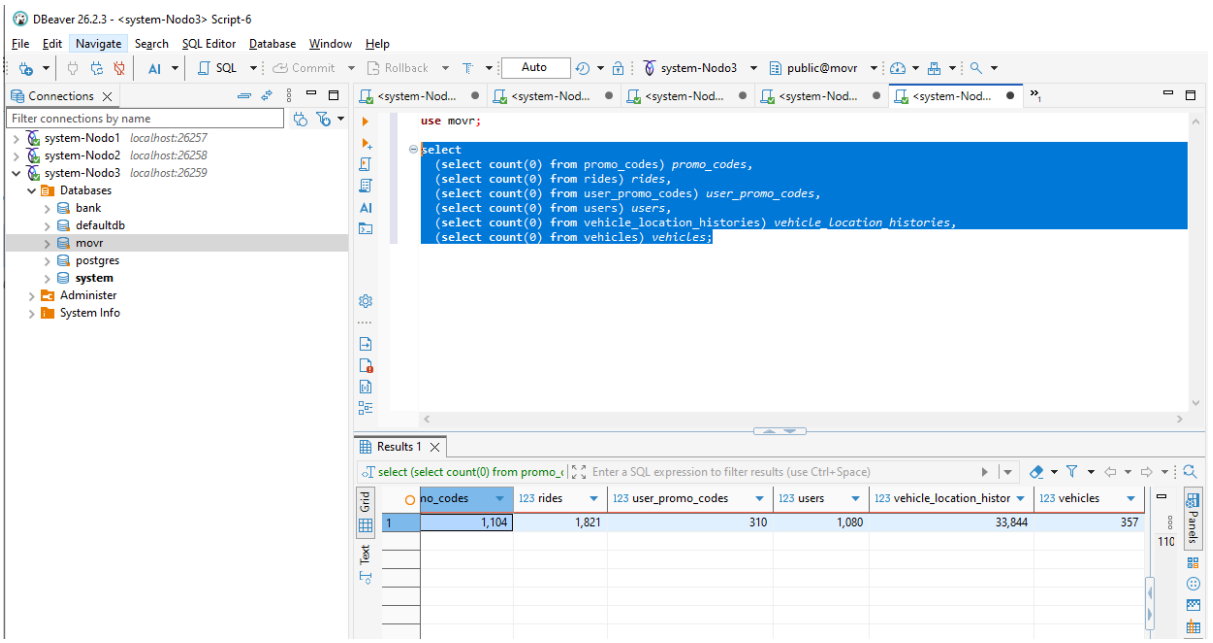
c. Ejecute el siguiente comando para ejecutar la simulación por 5 minutos.

```
PowerShell 7 (x64)
  17.0s        0            1.0            0.7      3.1      3.1      3.1      3.1 endRide
  17.0s        0          239.9          204.2      1.0      1.3      1.6      1.6 readVehicles
  17.0s        0            5.0            4.8     11.0     11.0     11.0     11.0 startRide
  17.0s        0           13.0           12.2     52.4     54.5     56.6     56.6 updateActiveRides
  18.0s        0            1.0            3.6     14.7     14.7     14.7     14.7 addUser
  18.0s        0            0.0            1.2      0.0      0.0      0.0      0.0 addVehicle
  18.0s        0            0.0            1.3      0.0      0.0      0.0      0.0 applyPromoCode
  18.0s        0            0.0            0.3      0.0      0.0      0.0      0.0 createPromoCode
  18.0s        0            0.0            0.7      0.0      0.0      0.0      0.0 endRide
  18.0s        0          115.1          199.2      1.2      1.6      1.8      2.0 readVehicles
  18.0s        0            6.0            4.9     12.6     25.2     25.2     25.2 startRide
  18.0s        0            6.0           11.8     56.6    209.7    209.7    209.7 updateActiveRides
_elapsed___errors__ops/sec(inst)___ops/sec(cum)__p50(ms)__p95(ms)__p99(ms)_pMax(ms)
  19.0s        0            2.0            3.5      3.8     22.0     22.0     22.0 addUser
  19.0s        0            1.0            1.2      7.6      7.6      7.6      7.6 addVehicle
  19.0s        0            0.0            1.2      0.0      0.0      0.0      0.0 applyPromoCode
  19.0s        0            1.0            0.3      3.4      3.4      3.4      3.4 createPromoCode
  19.0s        0            0.0            0.6      0.0      0.0      0.0      0.0 endRide
  19.0s        0           92.0          193.6      1.2      1.7      2.0      8.1 readVehicles
  19.0s        0            2.0            4.7     21.0     30.4     30.4     30.4 startRide
  19.0s        0            6.0           11.5    142.6    201.3    201.3    201.3 updateActiveRides
  20.0s        0            3.0            3.4      4.2      4.7      4.7      4.7 addUser
  20.0s        0            1.0            1.1     10.5     10.5     10.5     10.5 addVehicle
  20.0s        0            0.0            1.1      0.0      0.0      0.0      0.0 applyPromoCode
  20.0s        0            0.0            0.3      0.0      0.0      0.0      0.0 createPromoCode
  20.0s        0            0.0            0.6      0.0      0.0      0.0      0.0 endRide
  20.0s        0          236.9          195.7      1.2      1.8      2.2      6.3 readVehicles
  20.0s        0            3.0            4.6     13.6     14.2     14.2     14.2 startRide
  20.0s        0            7.0           11.3     62.9     79.7     79.7     79.7 updateActiveRides
```

d. Mientras la simulación se ejecuta, abra en el navegador la consola de administración del nodo 1 en el puerto 8080 utilizando la URL http://localhost:8080 y verifique en la pestaña de "Metrics" como cambian el gráfico en tiempo real por la ejecución de la simulación. Incluya una captura de pantalla.



e. Estando conectado a cualquier nodo, vuelva ejecute el siguiente script para consultar los datos de la base de datos “movr” que se acaba de modificar luego de la simulación. Incluya una captura de pantalla que muestre el resultado de la consulta.



Eliminación del Cluster

1. Ejecute los siguientes comandos para eliminar los contenedores y la red del ejercicio. Asegúrese de haber realizado completamente el laboratorio, ya que si ejecuta estos comandos debe volver a ejecutar los pasos nuevamente.

